ADR-2026-38

Unified Migration Authority

Status: Accepted | **Date:** 2026-03-11 | **Owner:** Daniel Shanklin / Director of AI & Technology

Related: ADR-2026-37 (DAG Validation), ADR-2026-04 (Medallion), SOP-005 (Release & Deployment)

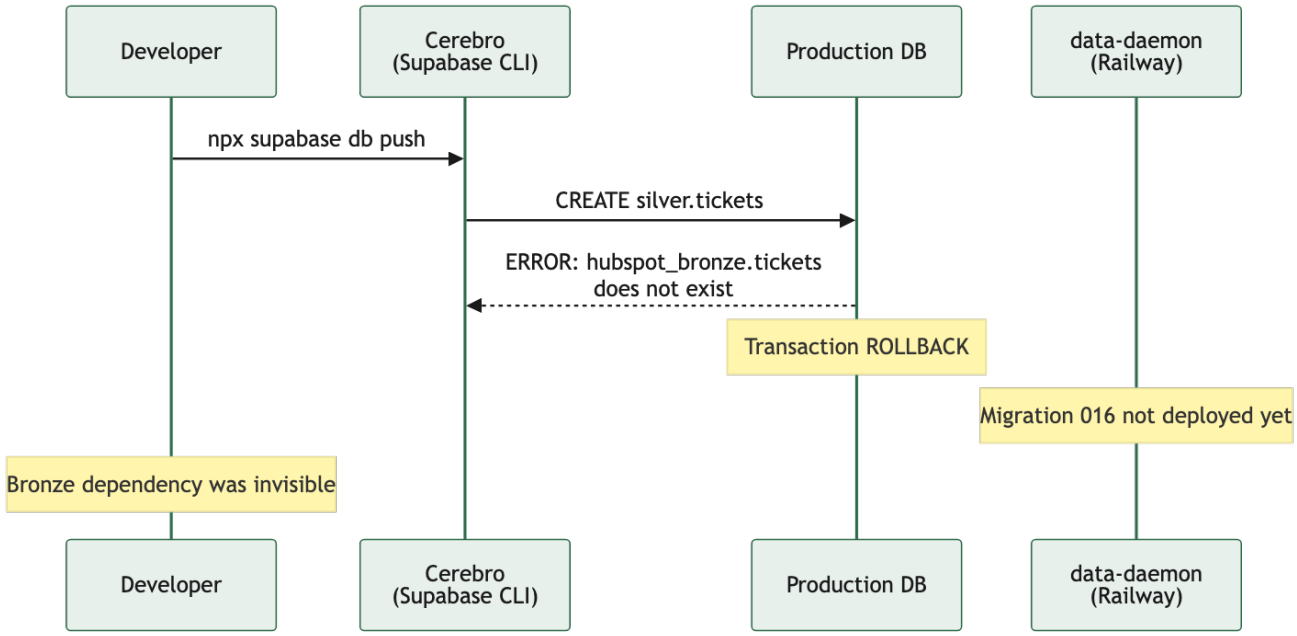
BLUF (Bottom Line Up Front)

All medallion schema DDL — bronze, silver, gold, RLS, forge — moves to a dedicated cerebro-migrations repo with Supabase CLI as the single migration tool. data-daemon and cerebro become pure consumers: one writes data, the other reads it. Neither owns the database. This eliminates cross-repo migration ordering failures by removing the cross-repo migration pattern entirely.

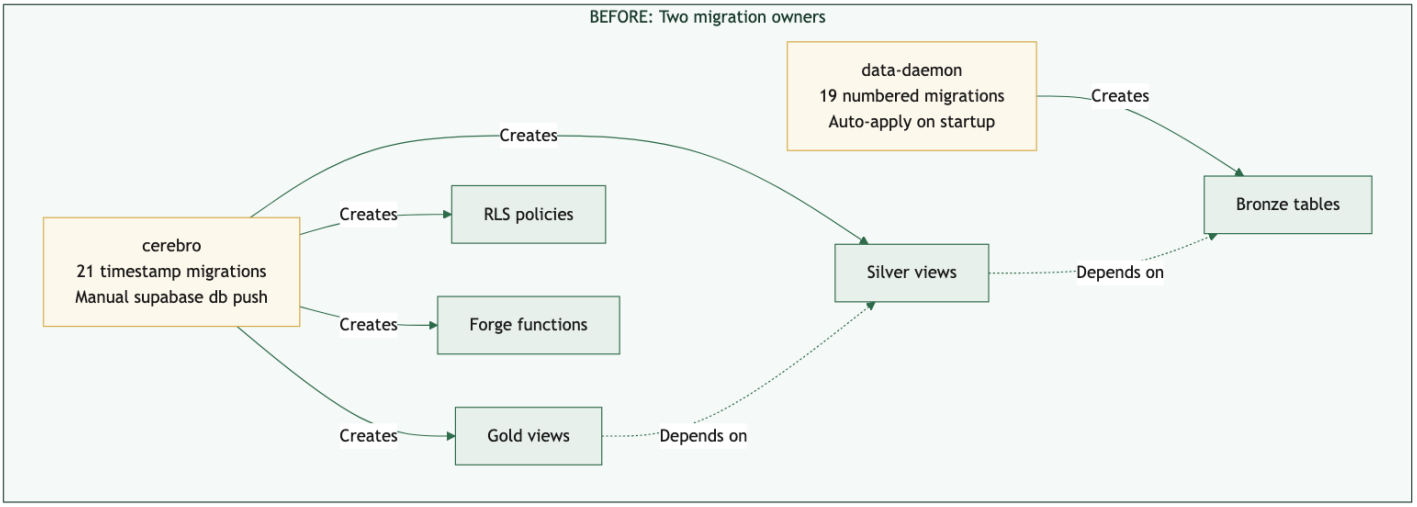
> **DECISION: Create a cerebro-migrations repo as the single migration authority for the shared Greenmark PostgreSQL database. All medallion DDL consolidates there. data-daemon and cerebro become pure consumers of the schema contract.**

Context: The Incident

On 2026-03-11, a cerebro silver expansion migration failed in production because it referenced hubspot_bronze.tickets — a bronze table created by data-daemon migration 016, which hadn't been deployed. Root cause: two repos independently mutating the same database with no coordination mechanism.

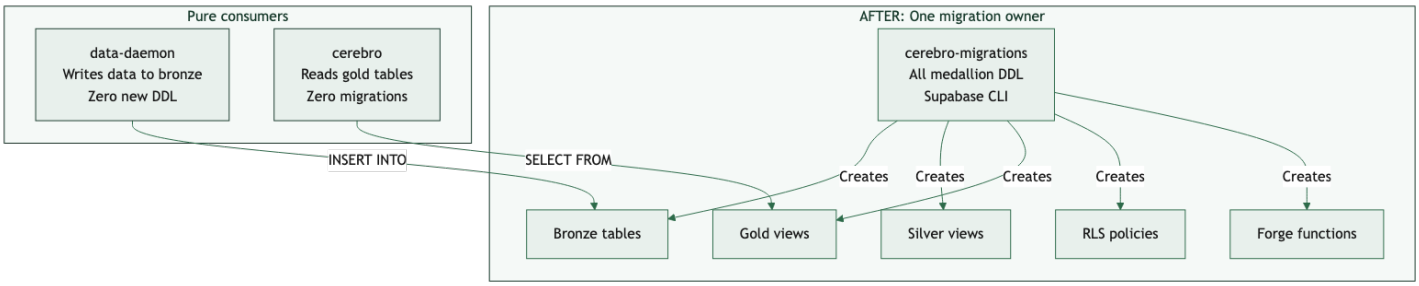


The Problem: Split Migration Ownership



Two repos, two migration systems, two history tables, one database. The dependency between them was invisible and unenforceable.

The Solution: Dedicated Migrations Repo



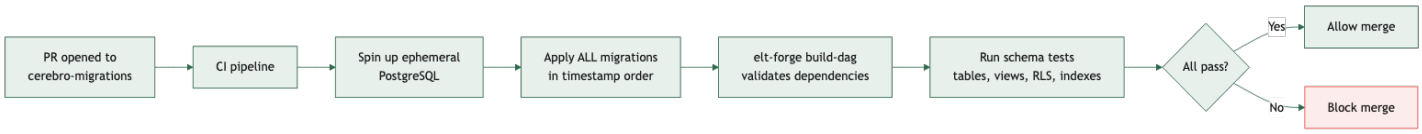
Service Roles After Migration

Service	Role	Owns
cerebro-migrations	Single migration authority	All medallion DDL (bronze, silver, gold, RLS, forge). Supabase CLI. CI validation.
data-daemon	Pure ETL runner	ETL connectors, sync logic. Daemon-internal tables only (daemon.migration_history, daemon.sync_state, daemon.job_queue). Frozen at migration 019.
cerebro	Pure frontend consumer	Next.js app, API routes, UI. Zero migrations. Reads gold tables via Supabase client.

What Moves to cerebro-migrations

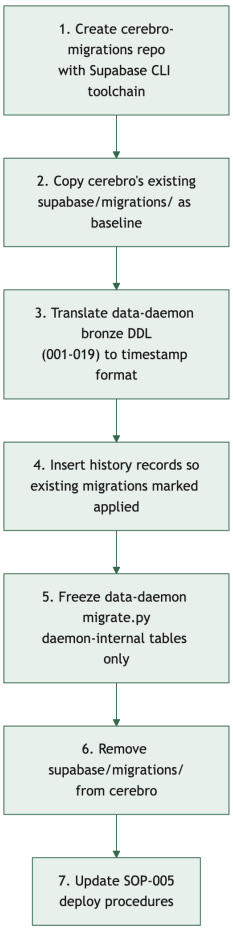
Artifact	Current Owner	Source
Bronze table DDL	data-daemon	Migrations 001-019
Bronze indexes/constraints	data-daemon	Migrations 001-019
Silver materialized views	cerebro	supabase/migrations/
Gold materialized views	cerebro	supabase/migrations/
RLS policies and grants	cerebro	supabase/migrations/
Forge refresh functions	cerebro	supabase/migrations/
Schema/role setup	cerebro	supabase/migrations/

CI Validation: Ephemeral Database



CI validates the full migration stack in isolation — no production queries, no point-in-time checks, no race conditions. Powered by elt-forge DAG validation (ADR-2026-37).

One-Time Cutover Plan



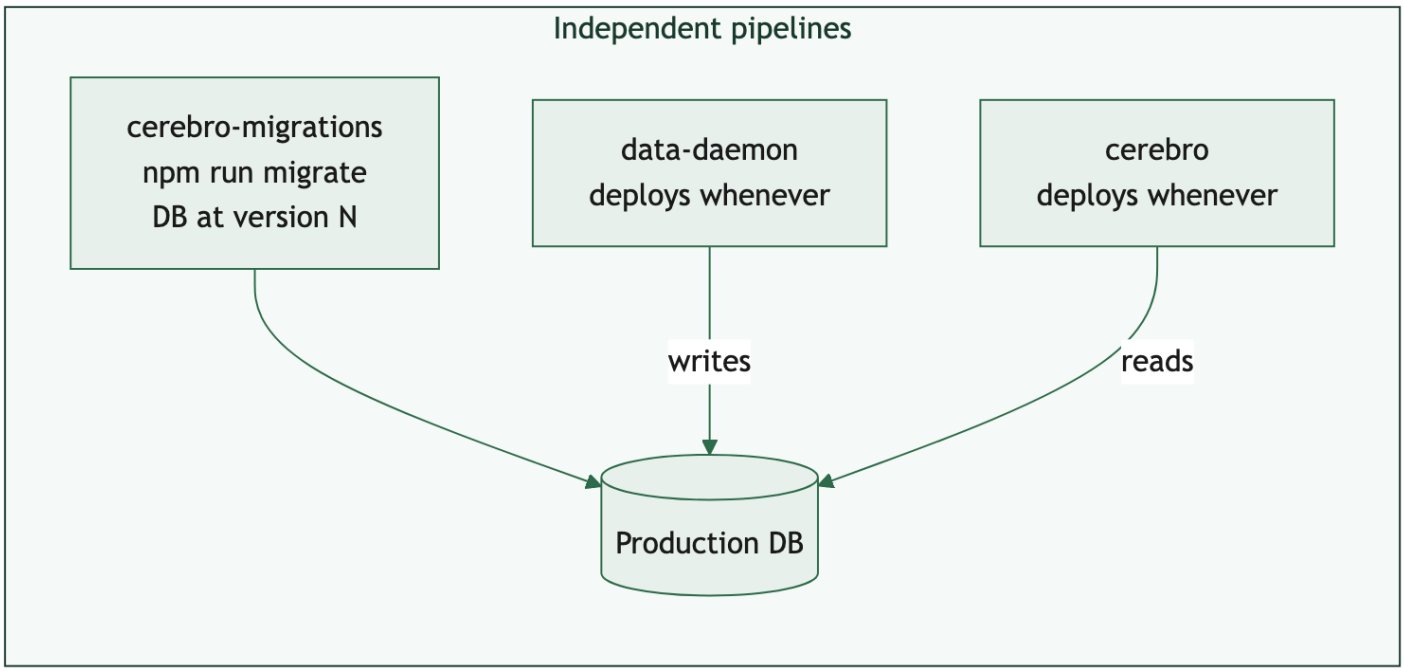
Options Considered (16 Evaluated)

We evaluated 16 architectural options across five criteria: incident prevention, simplicity for a 2-person team, scalability, AI agent compatibility, and clean ownership.

Tier	Options	Verdict
S Tier	Dedicated migrations repo + ephemeral DB CI	Accepted. Eliminates the problem structurally. Clean ownership.
A Tier	Consolidate into cerebro; Monorepo	Viable but semantic mismatch (frontend owns DB) or disproportionate blast radius.
B Tier	Reverse CI; Git submodule; Versioned artifacts; Deploy manifest	Overbuilt for a 2-person team.
C Tier	Header comments; Lock file; Contract tests; elt-forge registry	Brittle or indirect. Not worth the effort.
F Tier	Do nothing (SOP only); CREATE IF NOT EXISTS; CI queries production	Actively dangerous. False sense of security.

Deploy Coordination: Schema-First, Apps Tolerant

Schema repo and app repos have independent CI/CD. No cross-repo triggers. No orchestrator. No release branches. Schema deploys first, apps lag slightly behind.



Both apps are already forward-compatible: data-daemon ETL jobs fail gracefully if a table doesn't exist yet; cerebro pages show no data if a gold view is missing.

Breaking changes use expand/contract:

Step	What Happens	Risk
1. Expand	Migration adds the new thing alongside the old	Zero — additive only
2. Update	Apps update to use the new thing (separate deploys)	Zero — old thing still exists
3. Contract	Migration removes the old thing	Zero — no consumers remain

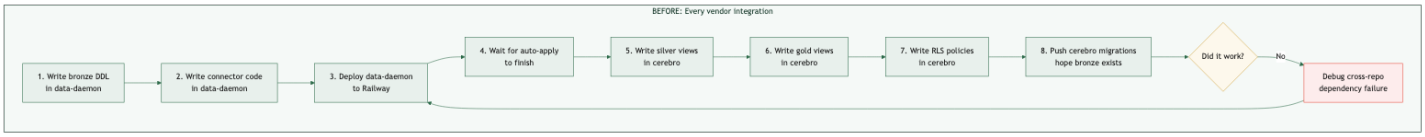
Three separate commits, three separate deploys, zero coordination needed. Each step is independently safe.

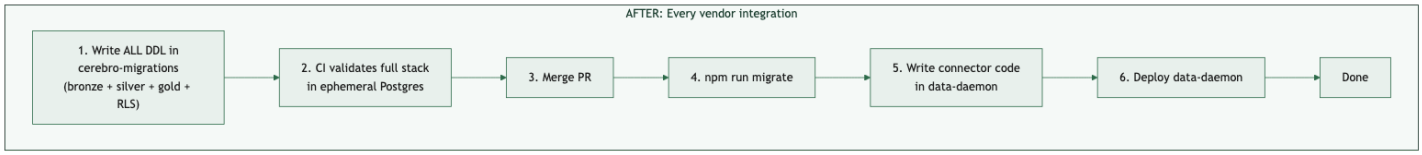
Rationale

Factor	Assessment
Industry consensus	Every source converges: one migration owner per shared database
Incident prevention	Cross-repo ordering failures structurally impossible — one repo
Clean ownership	Neither service 'owns' the database. The database has its own repo. Both services are consumers
AI agent workflow	Agent writes DDL in one place, runs one command. No cross-repo dependency reasoning
RLS safety	Policy changes go through PR review, not auto-applied on ETL startup
Scale	New vendor integrations need one PR — bronze + silver + gold in one sequence

Why We Had to Change

The split migration pattern creates compounding pain as the system grows. Every new vendor integration multiplies the coordination cost.





Projected Time & Hassle Savings (Next 12 Months)

Based on current trajectory: 5 new vendor integrations planned (Sage expansion, Navusoft expansion, fleet telematics, QuickBooks, ServiceTitan), plus ongoing HubSpot object additions. Each integration touches bronze + silver + gold + RLS.

Activity	Before (per integration)	After (per integration)	Annual Savings (5+ integrations)
Migration authoring	2 PRs, 2 repos, manual ordering	1 PR, 1 repo, natural ordering	~5 hours saved
Deploy coordination	Deploy daemon, wait, deploy cerebro, hope	npm run migrate, done	~5 hours saved
Cross-repo debugging	1-2 hours per ordering failure	CI catches before merge	~10 hours saved
Rollback complexity	Coordinate 2 repos, 2 systems	Revert 1 migration	~3 hours saved
Agent reasoning overhead	Cross-repo dependency analysis per deploy	Zero — one repo, one sequence	~8 hours saved
TOTAL			**~31 hours/year + eliminated incident class**

The time savings are real but secondary. The primary value is eliminating an entire class of production failures. The 2026-03-11 incident cost 3 hours of debugging plus delayed the HubSpot expansion launch. One prevented incident per quarter pays for the cutover effort in the first month.

Consequences

Positive: Cross-repo migration ordering failures eliminated. One source of truth for database schema. Clean separation: migrations repo owns schema, data-daemon owns ETL, cerebro owns UI. RLS changes get proper review. AI agents have simple workflow.

Negative: Third repo to maintain. One-time cutover effort to translate and merge existing migrations. Bronze DDL committed separately from connector code.

Mitigation: AI agents handle multi-repo workflows trivially — repo count is not a friction factor. One-time cutover is scriptable. CODEOWNERS on cerebro-migrations ensures review discipline.

When to Revisit

- If team grows beyond 5 engineers and migration review becomes a bottleneck
- If a third service needs to write DDL — validates this pattern
- If Supabase CLI gains native multi-repo migration support